LAB 1

Setting up your development environment

DUE DATE: 04/16/2026 11:59 PM

Lab Overview:

For this lab, we will be setting up our Linux development environment for systems programming in C. We will set up the environment in both **CS ix-dev** and a **virtual machine**. In this course, we will mainly use ix-dev and a virtual machine as an auxiliary.

Make sure to follow all steps outlined in the "tasks" subsection of this lab description. This setup should be done as early as possible. There may be many issues depending on your machine, and it is better to resolve them early rather than later. Be sure to ask questions if anything is not immediately clear.

Core Tasks:

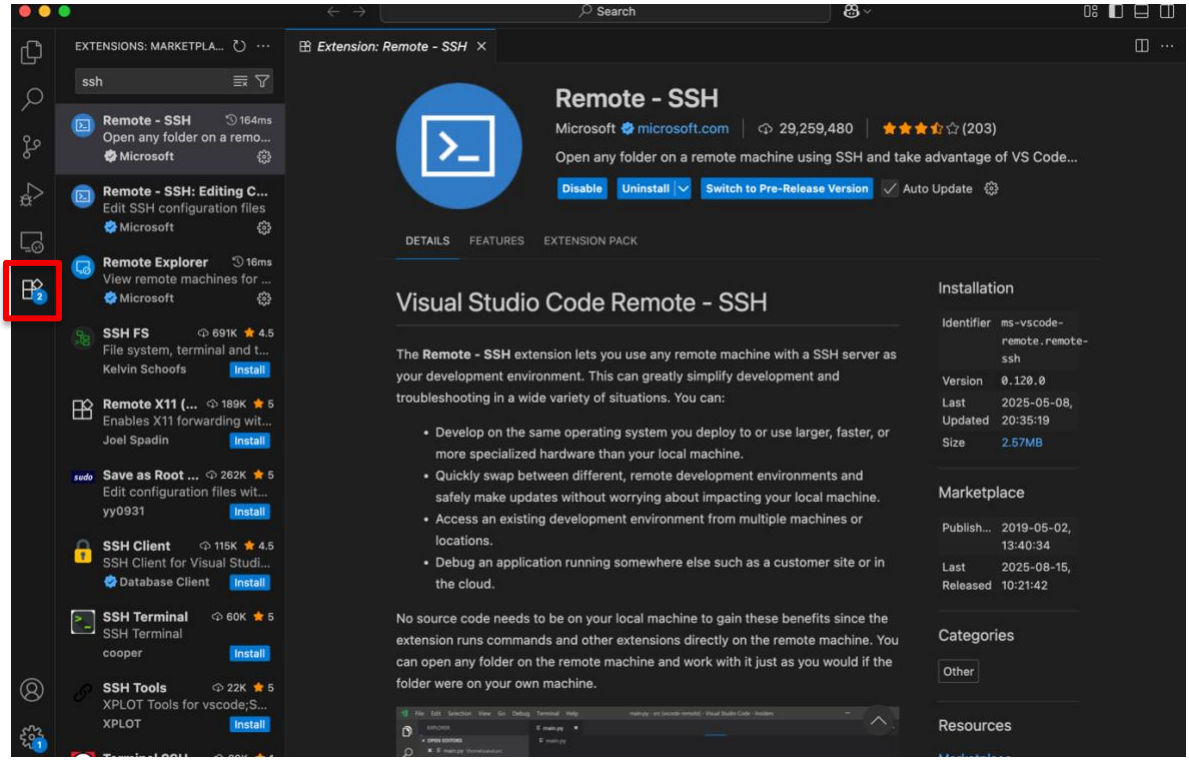
1. Connect the CS **ix-dev** server using SSH and test the C program compile and debug.
2. Set up your virtual machine (VM) and install Ubuntu 24.04.3 (Noble Numbat)
3. Install development tools (GDB, Valgrind, IDE)
4. Hello World in C. (developing, compiling, running, and testing code)

Connect to ix-dev using SSH:

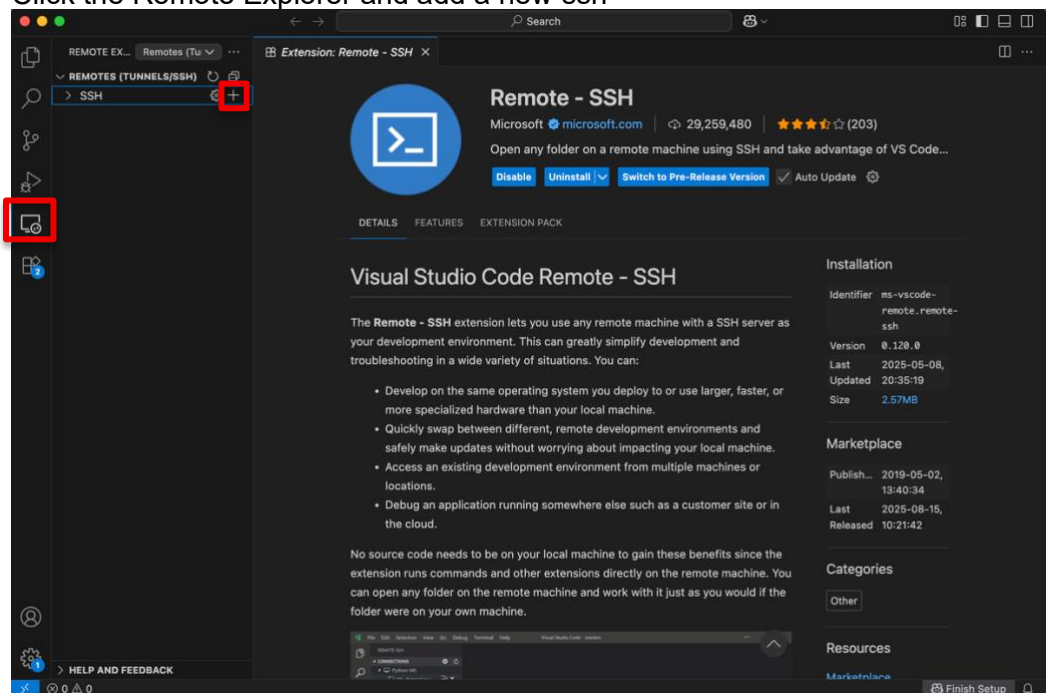
You can follow the instructions on this website (https://ix.cs.uoregon.edu/~norris/cis330/index.cgi?n=Main.ConnectingToIx) to connect to the ix-dev.

Another option to connect to the ix-dev using SSH is VS **Code**.

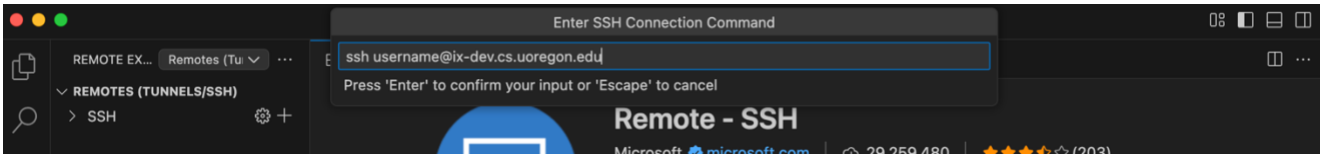
1. Download VS Code on your own computer (**NOT ON VM**)
    a. On macOS: https://go.microsoft.com/fwlink/?LinkID=534106
    b. On Windows: https://go.microsoft.com/fwlink/?LinkID=534107
2. Download the extension: **Remote – SSH**



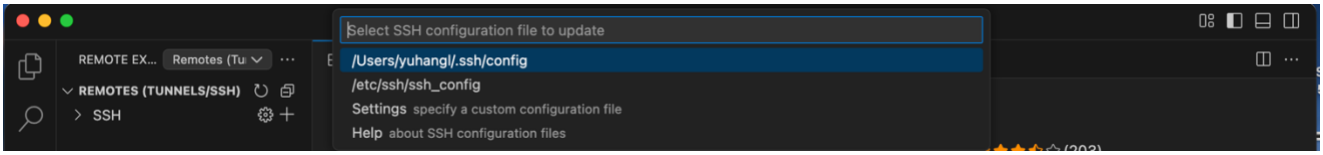
3. Click the Remote Explorer and add a new ssh

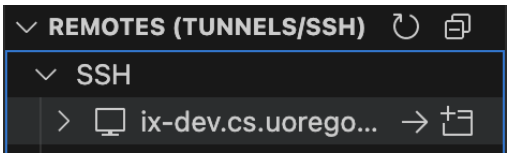


4. Type the following command: ssh username@ix-dev.cs.uoregon.edu. Replace the username with your CS username (may be different from your Duck ID).

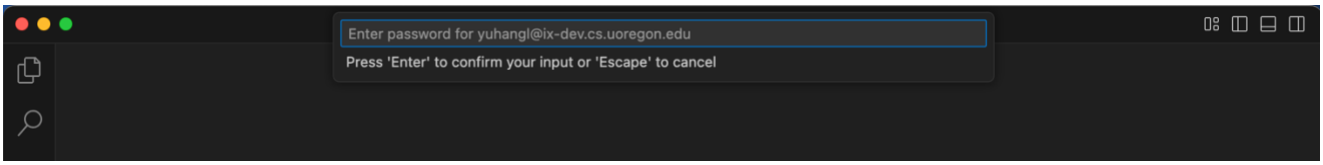


5. Click the first option and you will see the ssh is added to the list. Click the “arrow” to connect ix-dev.

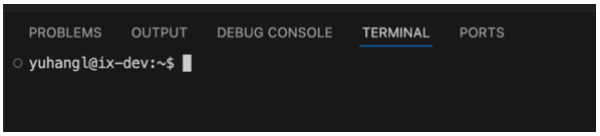




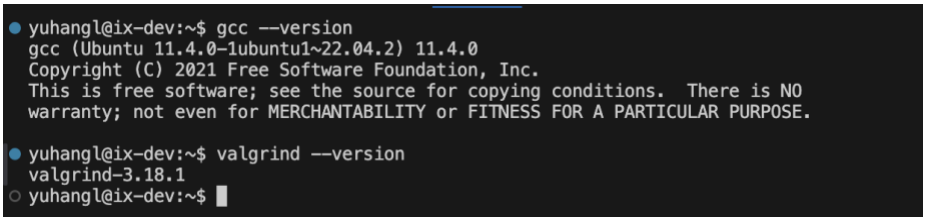
6. Input your password



7. If you login ix-dev successfully, your will see “username@ix-dev” on your terminal.



8. Test gcc and valgrind.



Set up Linux in VM:

Be sure to follow these instructions while setting up your VM. We will be grading your work on a VM running this exact setup, so if these instructions are not followed, then there is a significant chance that your submissions will not compile or run when we grade them. This is the most

important lab because the rest of your lab and project submission depends on it.

Important note:

A BIOS setting is required for Windows users. Check out this link to change your BIOS settings if your VM does not run. https://www.youtube.com/watch?v=m7cHUNXB7j4

For Mac users, go to Security&Privacy and check "detailed" shows below "Allow apps downloaded from" and restart your computer.

UTM: Another option for Mac users is UTM, which can be downloaded for free from the official UTM website. Once you have downloaded the AMD image and UTM, you can use UTM to then emulate the image, and then you will be prompted in a shell with a couple of options, but pick install ubuntu, and you should be good to go. NOTE: if you get an error "Display Output is not active," follow these instructions https://github.com/utmapp/UTM/discussions/5555. Also, this runs extremely slow.

Downloading files (VM).

1. Download and install Oracle Virtual Box.
    a. Link (https://www.virtualbox.org/)
2. Download the Linux ubuntu 24.04.3 (Noble Numbat) ISO
    a. Link
        1. If you are a **Windows/macOS user** whose laptops are regular **Intel-based** machines, downloading the "64-bit PC (AMD64) desktop image". Using following link: https://releases.ubuntu.com/noble/
        2. If you are using a MacBook with **Apple silicon**, downloading the "64-bit ARM (ARMv8/AArch64) desktop image". Using following link: https://cdimage.ubuntu.com/releases/noble/release/

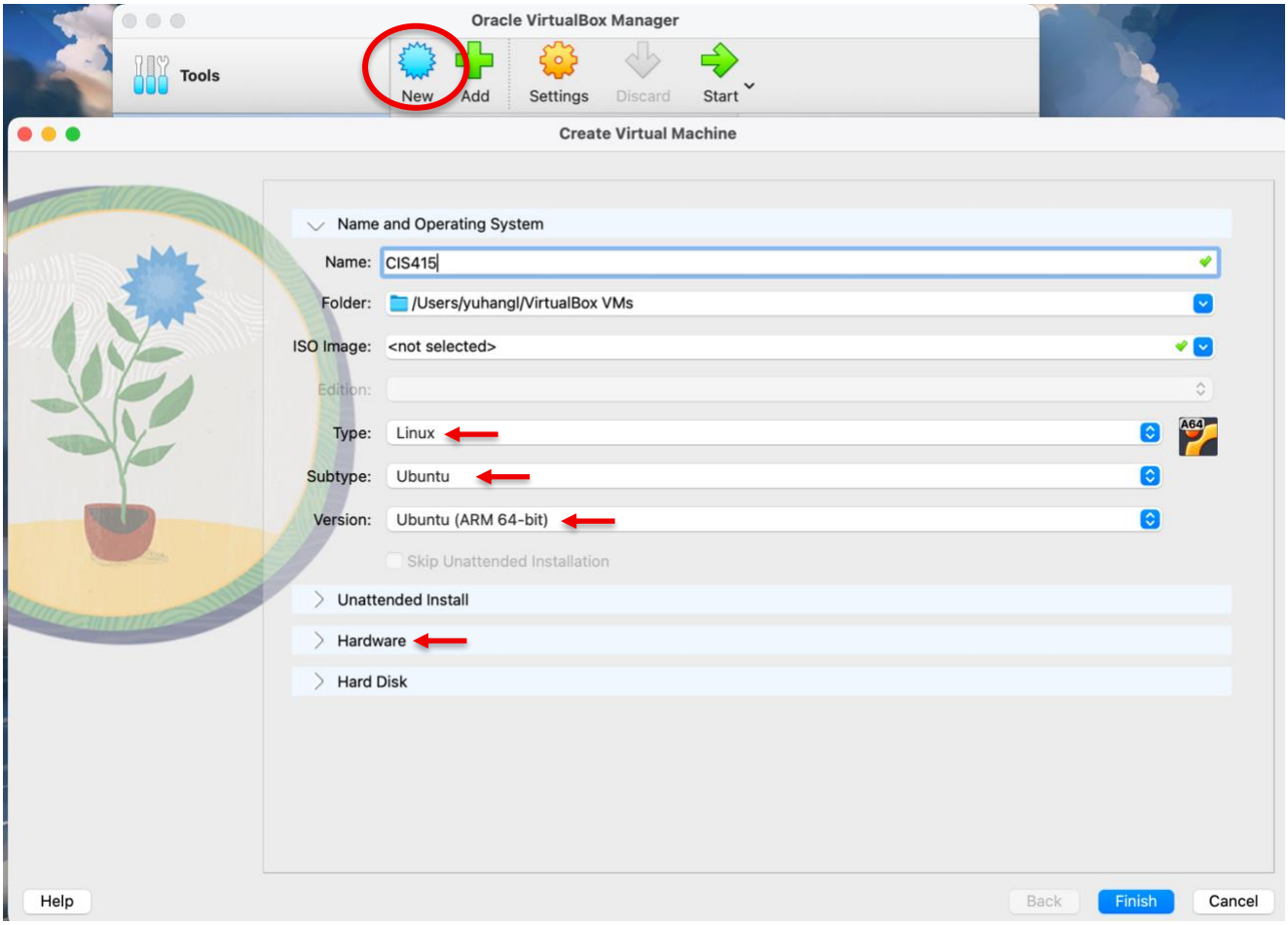
    b. Make sure you download the Desktop version

Setting up your VM.

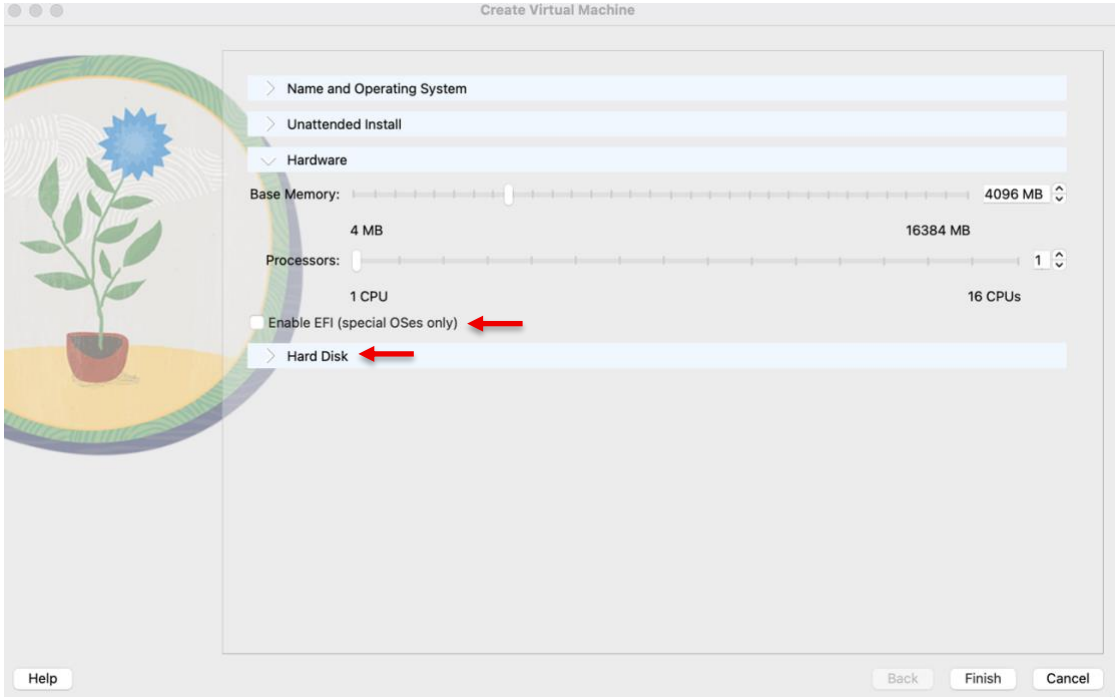
(Most of the instructions are for Mac; there should be only some minor differences compared to Windows 11)

Once you have the ISO downloaded and VirtualBox installed, open VirtualBox.

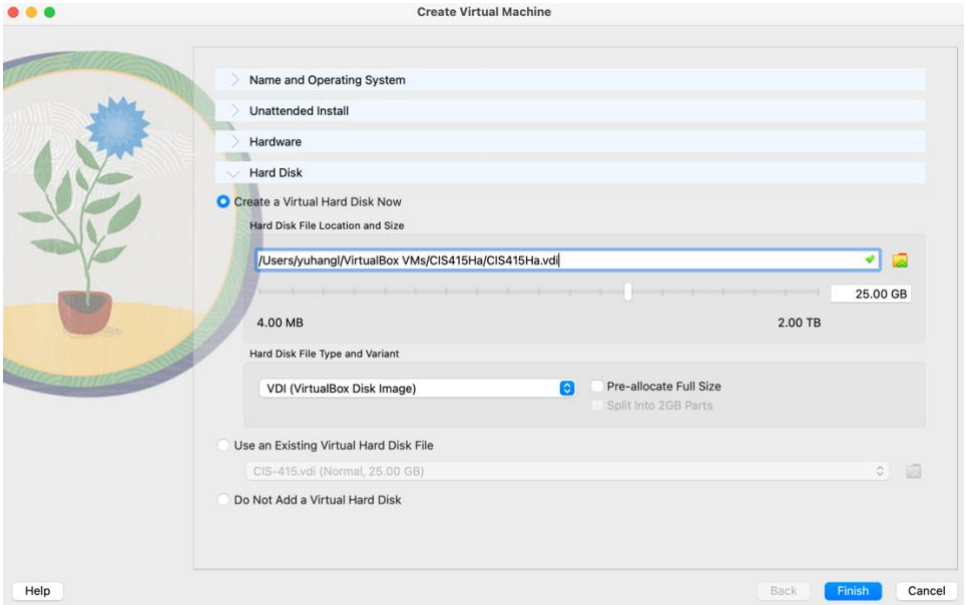
- Click on "New", then enter the name "CS-415".
- Change the machine folder to a location of your choice (make sure you have at least 15GB of space).
- The Type is Linux.
- The version is Ubuntu (64bits) (If you have a 32-bit machine which is very unlikely, please contact us)
- Click on Hardware



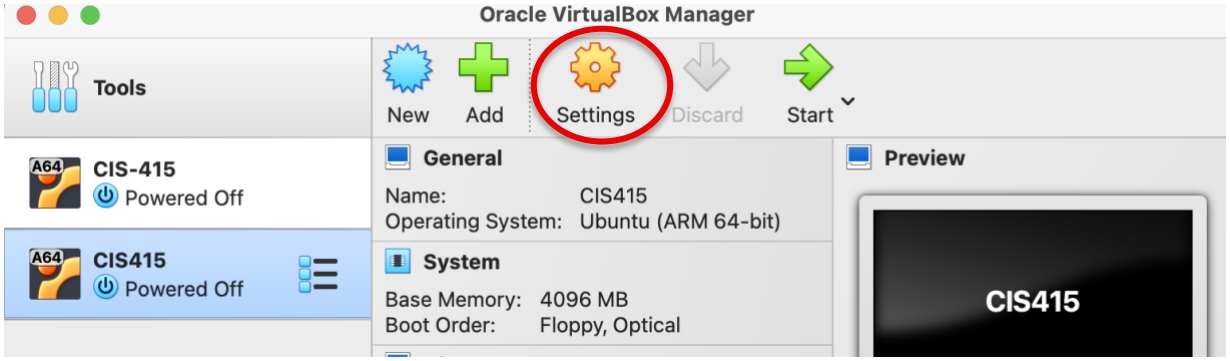
- Select the amount of RAM to use (we used 4GB of RAM. It's best to use a good amount so the VM runs smoothly, so if you have more to spare, you may add more. And unclick "Enable EFI". Click on "Hard Disk".



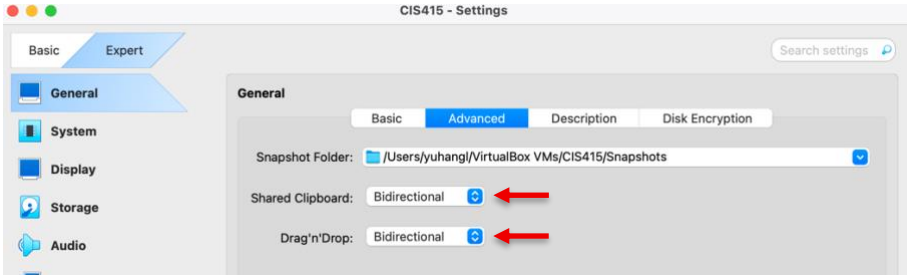
- Check the “create virtual disk now” (which should be the default), then click “create”.
- You should get three pop-up windows
    - Select “VirtualBox Disk Image.”
    - Select “Dynamically allocated”.
    - Assign 25 GB of memory. (Ubuntu requires at least 10G+ of storage space to install, so make sure you give the VM enough memory)
    - Click Finish.



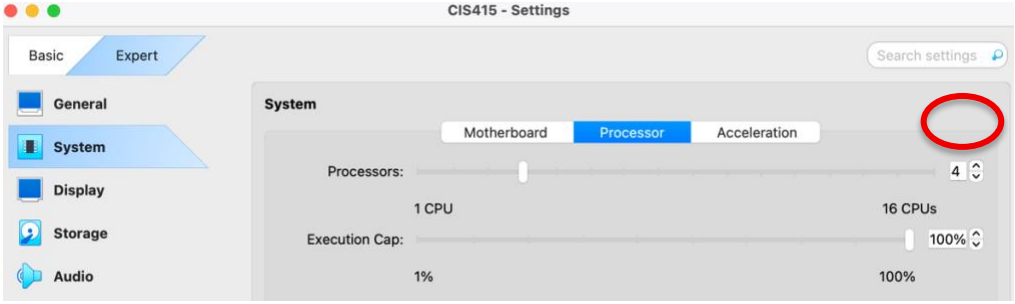
Settings for your VM



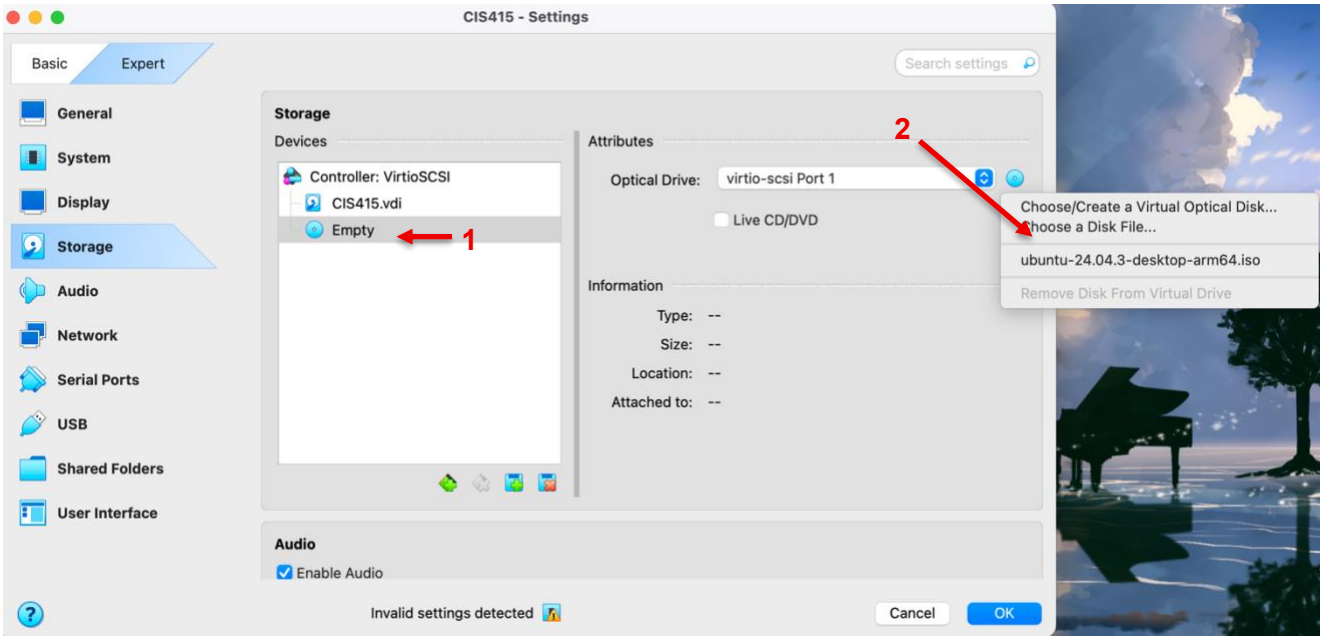
- Go to the General, Advanced tab
    - Change both “Shared Clipboard” and “Drag’ and ’Drop” to bidirectional



- Go to System, Processor tab
    - Change processor to at least 4 (this will be important for the last project)

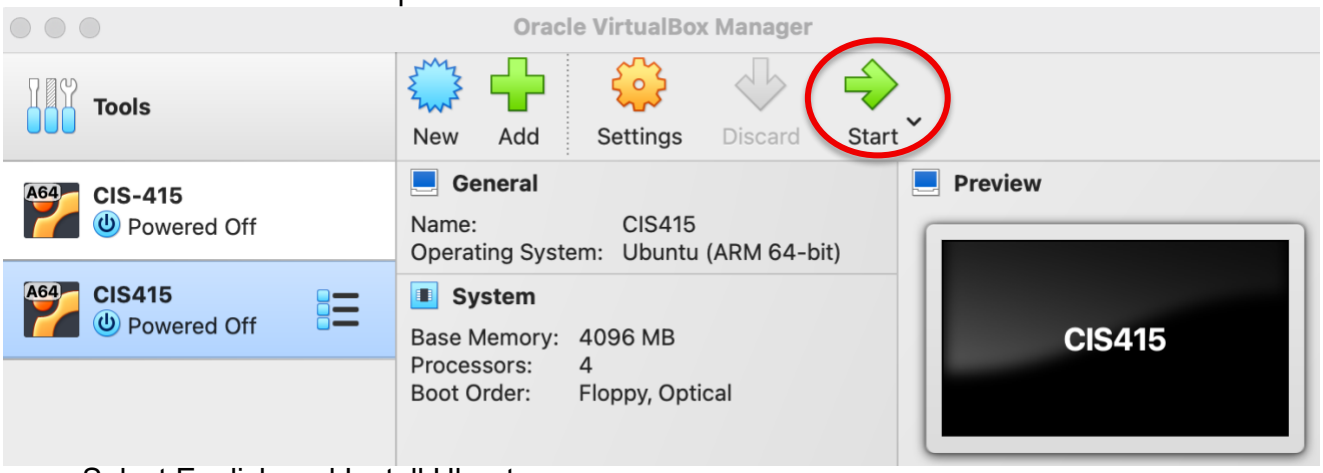


- Next, go to Storage
    - Click on the disk at location 1
    - Then edit the iso image you want to choose at location 2
    - Select the ubuntu 24.04.3 image you downloaded earlier

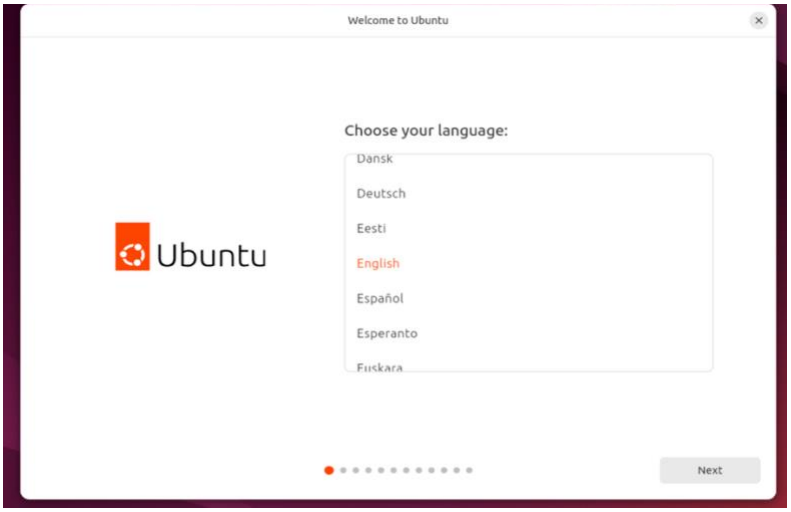


Now, Start your VM and Install Ubuntu

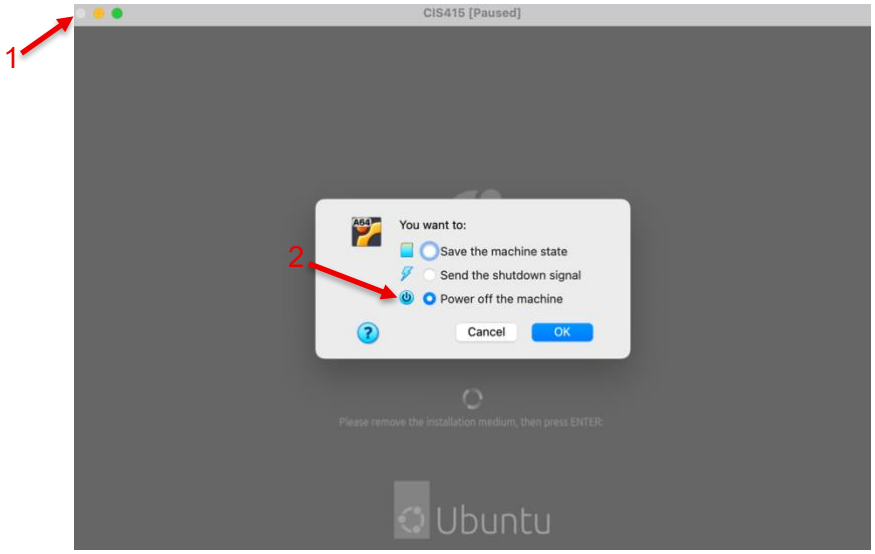
- Select your VM on the left-hand side and click “Start.”
  - First time boot up will take a little while



- Select English and Install Ubuntu



- For the next few windows, select the following options
  - Keyboard layout
    - English (US), English (US)) You can add other keyboard installations afterwards if you want to.
  - Connect to the internet
    - Use a wired connection
  - Updates and other software
    - Skip update
    - Select “Install Ubuntu.”
    - Select “Interactive installation.”
    - Select “Default selection” (or “Extended selection”)
    - Check “install third-party software …”
  - Installation type
    - Check and only check “Erase disk and install Ubuntu.”
  - Put in your name, username, and a password
  - Choose a location
  - Click on install now
- The installation will take a little time, so just be patient. Wait and relax.
- After installation, click “Restart Now”.
- When you see this window, “Please remove the installation medium, then press ENTER.”
  - Close the window at location “1.”
  - Select “power off the machine” at location “2.”
  - Click “OK.”



- Boot up your VM
  - If a window pops up and reminds you to upgrade, decline.
  - ***Don't upgrade! ***
- Let's get the system up to date
  - Press "control+shift+t" or right click on the desktop and click "Open Terminal"
  - Type in the following commands (using sudo commands will prompt a password request)
    - $ sudo apt update
    - $ sudo apt upgrade
  - Wait for the update to finish (you can change your window size by going into ubuntu settings, search "Display", and then change the resolution)

Extra tip on setting up shared folder
https://askubuntu.com/questions/456400/why-cant-i-access-a-shared-folder-from-within-my virtualbox-machine
You can follow this link to install the VBox guest version and setup a shared folder, this is not required.

Setting up our development environment:

- Bring up the terminal and install development tools with the following commands
  - $ sudo apt install build-essential
  - $ sudo apt install valgrind
- To check if you have successfully installed the development tools, use the following command
  - $ gcc --version
- If the terminal prompt "command 'gcc' not found," then you have to do the above step again.

IDE
You may use any IDE you like. Examples are VI, Eclipse, VSCode, etc.

Visual Studio Code:

- Download the installation file from https://code.visualstudio.com/Download. If you are a **Windows/macOS user** whose laptops are regular **Intel-based** machines, please download the **.deb** file with option **x64**. If you are using a MacBook with **Apple silicon**, please download the .deb file with option **Arm64**
- Run **'sudo apt install ./<file>.deb'** at the directory you download files **(Please substitute <file> with your actual file name)**
- After installation, you can use command **'code'** to open it via terminal and you can open visual studio code successfully
- You can also find official instruction in the following link: https://code.visualstudio.com/docs/setup/linux

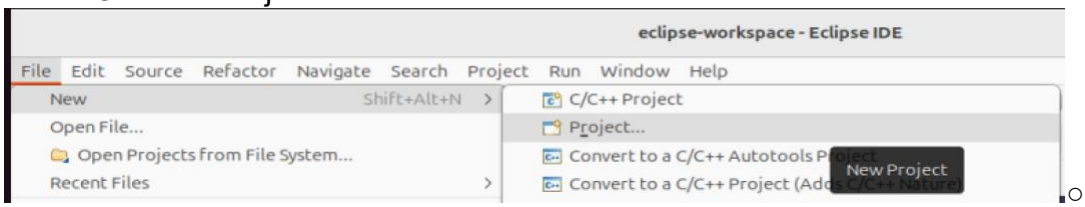
Some tips on **Eclipse** (optional)

Eclipse contains built-in GDB GUI and Valgrind which is convenient for developing C projects.

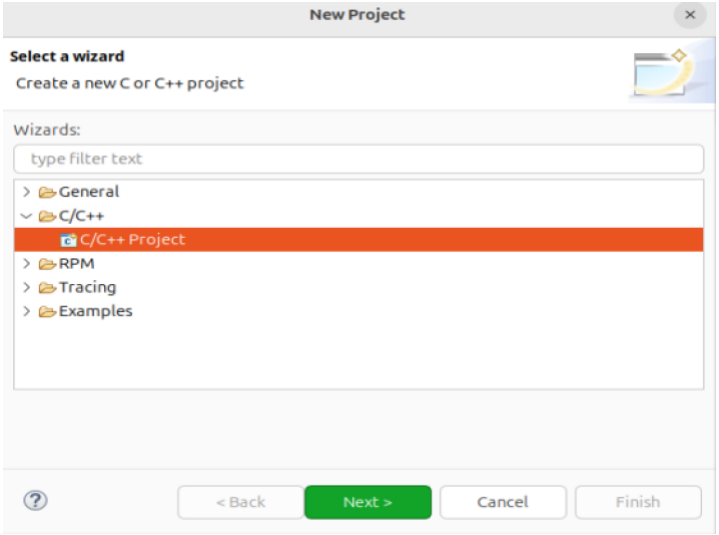
- Installing Eclipse
  - Go to link and download “Eclipse IDE 2025-03” ()
    (https://www.eclipse.org/downloads/packages/release/2025-03/r)
    - Select “Eclipse IDE for Embedded C/C++ Developers”
    - For AMD64, click: Linux: x86_64
    - For ARM, click: Linux: AArch64
  - Extract the tar file, open the directory and double click “eclipse”

Now let’s write a basic C program

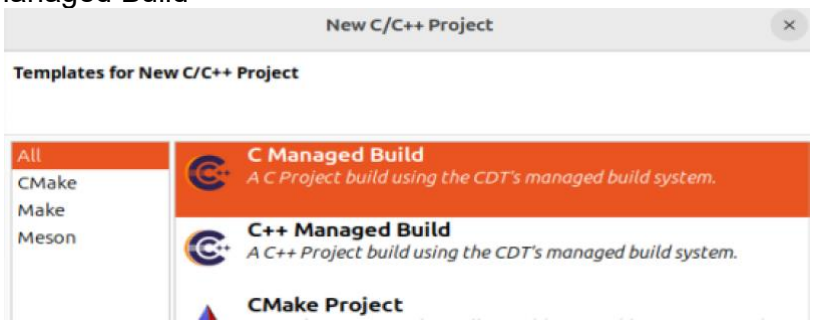
- Start your Eclipse
  - Click on “File”
  - Choose “Project…”



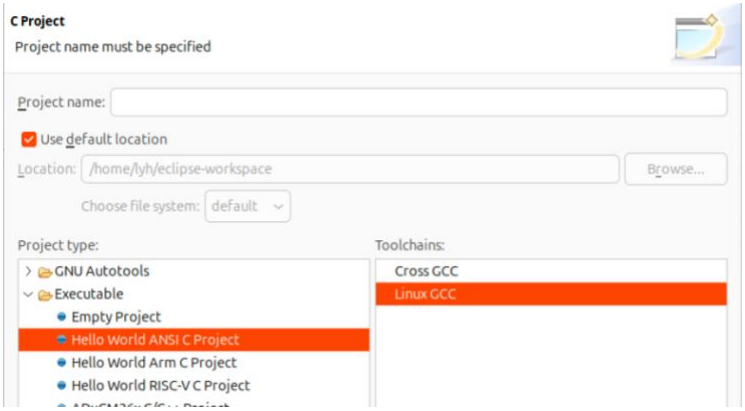
  - Choose “C Project”, “Click Next >”.



- Select “C Managed Build”



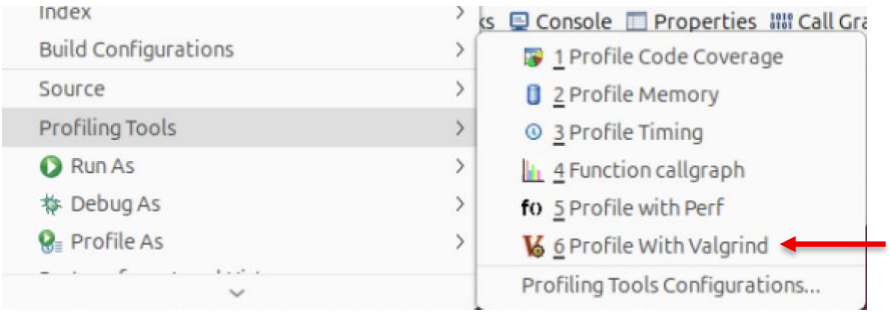
- Put “Lab1” in project name
- Choose “Hello World ANSI C Project” and “Linux GCC” on right



- Then click “Finish”
- Click on the hammer icon on top to build
- Click on the green play icon to run

● You can also run “valgrind” to check memory leak in Eclipse

- Right click on project “Lab1”
- Select “profiling Tools”
- Profile with Valgrind



● Eclipse will generate a default Makefile for you. Please write a Makefile for your project. ○ For future projects and labs, you will also be writing your own Makefiles ● ***Your target executable should be named "Lab1.out"***

SUBMISSION:

- What to Submit
  - A tar or zip file including:
    - Lab1.c (this file only prints hello world)
    - Makefile (Your target executable should be named "Lab1.out")
- The naming convention of the zip/tar file while uploading to Canvas

  UoID_duckID_LAB/ProjectX
    - UOID: alex
    - DuckID: 951505xxx
    - Submission for: Lab1
    - So the name of the zip/tar file should be:
      - alex_951505xxx_Lab1.tar or alex_951505xxx_Lab1.zip
- How We are Going to Test Your Program
  - Open a terminal where your source code is
  - Type command (your src code and Makefile must be in the same directory)
    - $ make
    - $ ./Lab1.out
  - To test for memory leak type
    - $ valgrind ./Lab1.out